



Trailhead



[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



[Write Efficient Queries](#) ▼

Write Efficient Queries

Learning Objectives

After completing this unit, you'll understand:

- How the Salesforce Query Optimizer optimizes query performance.
- The impact that using selective queries has on query performance.
- How to use the Query Plan tool to evaluate search queries.

Limits

Efficient queries not only perform better, but they help ensure that you don't run into problems with governor limits. Remember, this is a multi-tenanted platform where everyone has to share space. And the No. 1 thing that can crash a system faster than anything is a poorly performing query. And that is where governor limits come in.

Think of governor limits as a kind of resource referee. They ensure that everyone is playing by the rules and receives an equal share of the resources pie.

In this unit, you're going to learn what you can do to optimize your search queries to avoid ever having to be limited by the limits.

The Salesforce Query Optimizer

Salesforce's back-end database uses Oracle, but Salesforce uses its own version of a Query Optimizer to evaluate cost-based queries. Like most cost-based query optimizers, the one that Salesforce uses relies on statistics gathered about the data. Most statistics are gathered weekly, but the system also generates pre-queries that are cached every hour.

The Query Optimizer evaluates SOQL queries and SOSL searches. It acts as a sort of traffic cop by routing queries to the appropriate indexes. It looks at every incoming query and assigns a cost value for each potential query path that it identifies. It then uses these costs to determine which execution plan to use.





Best Practices

~~Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)~~
We know .NET developers love to think about best practices. You love to challenge yourself and always look for the best way of doing things. We get that. So we thought you would love to learn which best practices to use to build fast and efficient search queries.

Write Efficient Queries

Building Selective Queries

In the first unit we covered SOQL statements and how you can apply filters using the WHERE clause. You can even combine multiple fields using AND and OR clauses.

Well, and we know you won't be surprised to hear this, having more fields in your WHERE clause is a good thing. Obviously, the less data that your query returns, the better. But what you might not know is that not all fields are the same. Some fields are what you can think of as "power" fields. If you use these power fields in the WHERE clause, your queries are super-duper fast.

And what makes these power fields so powerful? Indexes, of course.

For all standard and custom tables, certain fields are automatically flagged to be indexed. These fields include the following:

Indexed Field	Description
Id	Unique 18-character field that is system generated. This is the primary key for the object.
Name	Text-based field.
OwnerId	Reference to the owner of the object.
CreatedDate	Date and time when the record was created.



	This field is indexed where the similar LastModifiedDate is not, so consider using this one in your queries.
Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)	
RecordType Write Efficient Queries	Id of the RecordType. RecordTypes are used to offer different UI results to certain users.
Master-Detail Fields	Foreign key field used to indicate a master-detail relationship
Lookup Fields	Foreign key field used to indicate a lookup relationship
Unique Fields	Custom fields can be marked as unique when they are created, and this will automatically make them indexed.
External ID Fields	Like unique fields, these custom fields can be marked as an External Id and are mainly used for integration purposes.

Anytime you use one of these indexed fields in your query's WHERE clause, you're increasing the chance that your query is considered selective and an index used as opposed to a full table scan. We can't stress enough how important this is when you're dealing with large databases.

Note: It's possible for Salesforce customers to request a custom index from Salesforce Support by creating a support case and including the SOQL query with the indexed field.

Index Selectivity Exceptions

Using an indexed field in your query doesn't always make it golden. You can do things in your queries to make them non-selective and thus prone to the dreaded full table scan. When building your queries, always strive to avoid these things.

- **Querying for null rows**—Queries that look for records in which the field is empty or null. For example:

```
SELECT Id, Name FROM Account WHERE Custom_Field__c = null
```





```
SELECT Id, LastName, FirstName FROM Contact WHERE LastName LIKE '%smi%' copy
```

• **Text fields with comparison operators**—Using comparison operators, such as >, <, Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

```
SELECT AccountId, Amount FROM Opportunity WHERE Order_Number__c > 10 copy
```

Write Efficient Queries

Query Plan Tool

Our friend the Developer Console contains a neat little tool to speed up your queries. It gives you a behind-the-scenes peek into how the Query Optimizer works. The Query Plan tool isn't enabled by default. Enable it by doing the following.

1. From the Setup menu, select **Developer Console**.
2. In the Developer Console, select **Help > Preferences**.
3. Select **Enable Query Plan** and make sure that it's set to true.
4. Click **Save**.
5. In the Query Editor tab, confirm that the Query Plan button is now next to the Execute button.

Now that the Query Plan tool is enabled, you can use it to evaluate queries. Let's start by using it to evaluate a poorly performing query.

1. In the Developer Console, click the **Query Editor** tab.
2. Delete any existing query, and insert the following query:

```
SELECT Id, CaseNumber FROM Case WHERE Status != 'New' copy
```

3. Click **Query Plan**.

The Query Plan dialog displays a table that lists the cost of the query and that it will be doing a TableScan. This is not a good thing. Check out the notes in the bottom pane, which further explain the results.

Now let's look at another query that gets the same results as the first query.

1. Click the **Query Editor** tab in the bottom pane.
2. Delete the existing query, and insert the following query:

```
SELECT Id, CaseNumber FROM Case WHERE IsClosed = true copy
```

3. Click **Query Plan**.

The Query Plan dialog displays a table that lists the cost of the queries, but this time you see another row displayed indicating that it's possible to use an index.



For more information about what each column in the Query Plan tool reveals, see the [Database Basics: Writing Efficient Queries \(dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/database_basics_dotnet/writing_efficient_queries?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

Write More

Always strive to avoid creating queries with non-deterministic formula fields. Formula fields are custom fields that allow you to dynamically calculate a field's value at runtime. These types of fields tend to be popular in most orgs and can easily be overused. A formula field is considered non-deterministic when its value varies over time. Check the link in Resources about SOQL best practices: Nulls and Formula Fields for more info.

Unfortunately, you can't use the Query Plan tool to evaluate SOSL queries, but that doesn't mean that you don't consider performance with those types of queries. Learn more about what to avoid when writing SOSL queries by referring to the Best Practices and Performance Tips links referenced in Resources.

Resources

- [Salesforce Help: Lightning Platform query optimization FAQ](https://help.salesforce.com/s/articleView?id=000386021&type=1)
(<https://help.salesforce.com/s/articleView?id=000386021&type=1>)
- [Salesforce Help: Make SOQL Query Selective](https://help.salesforce.com/s/articleView?id=000385218&type=1)
(<https://help.salesforce.com/s/articleView?id=000385218&type=1>)
- [Salesforce Help: Developer Console Query Plan Tool FAQ](https://help.salesforce.com/s/articleView?id=000386864&type=1)
(<https://help.salesforce.com/s/articleView?id=000386864&type=1>)
- [Salesforce Developers Blog: SOQL Performance Tips: LastModifiedDate vs SystemModStamp](https://developer.salesforce.com/blogs/engineering/2014/11/force-com-soql-performance-tips-systemmodstamp-vs-lastmodifieddate-2)
(<https://developer.salesforce.com/blogs/engineering/2014/11/force-com-soql-performance-tips-systemmodstamp-vs-lastmodifieddate-2>)
- [Salesforce Developers Blog: SOQL Best Practices: Nulls and Formula Fields](https://developer.salesforce.com/blogs/engineering/2013/02/force-com-soql-best-practices-nulls-and-formula-fields)
(<https://developer.salesforce.com/blogs/engineering/2013/02/force-com-soql-best-practices-nulls-and-formula-fields>)

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points



A CreatedDate

[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

B Name

Write Efficient Queries

C RecordType

D External ID Fields

E All of the above

2 When building selective queries always strive to avoid:

A Querying for null rows

B Using text fields with comparison operators

C Querying with LIKE operators

D A and B

E A and C

Second attempt earns 50 points. Three or more earns 25 points.



Learn	Certifications	Community	Extras
Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)			
Trails	Maintain Certifications	Events	Trailhead Login
Write Efficient Queries	Verify Certifications	Agentblazers	Sales Enablement
Trailmixes		Salesforce Developers	Trailblazer Store
Superbadges		Salesforce Admins	Salesforce Help
Trailhead Academy		Code of Conduct	
Career Paths		Report Illegal Content (EU)	

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

Engli

Your Privacy Choices